AgentAudit: How Much of an Agent Leaderboard Is Signal?

Francisco Javier Gómez Pulido *fran@aapex.io*
Aapex, Seville, Spain

Abstract

Agent leaderboards report one number per system from a single run. We ask how much of that number is signal. We introduce AgentAudit, a framework for statistically rigorous evaluation of LLM agents (unbiased pass@k and pass^k, intervals that respect the tasks-by-runs structure, variance decomposition, paired tests with multiplicity control, ranking uncertainty, a power planner and anytime-valid sequential stopping) together with AgentAudit-Bench, an offline, deterministic, state-verified benchmark of 360 procedurally generated tool-use tasks with seeded fault injection. A simulation study with agents of known skill shows that the common shortcut of pooling all runs gives 95% intervals that cover only 68.3% of the time with ten runs per task, and tests that pool runs reject a true null in 17.9% of simulations (nominal 5%); task-level procedures stay near nominal. At a fixed attempt budget, more tasks beat more repeats. Re-analysing 129 public SWE-bench Verified submissions, 28.3% of agent pairs are statistically indistinguishable after multiplicity correction, and *none* of the 128 adjacent leaderboard positions is distinguishable. On tau-bench, the gap between two frontier models is significant in retail but not in airline. On 7 local models (10 runs on 120 tasks) run-to-run noise accounts for 34% of score variance and a 5-point difference would need about 387 tasks.

1 Introduction

Public leaderboards for LLM agents, from SWE-bench (Jimenez et al., 2024) to tau-bench (Yao et al., 2024) and the Holistic Agent Leaderboard (Kapoor et al., 2026), rank systems by a single success rate obtained from one run per task. Claims such as "new state of the art" are made on differences of one or two points. Three sources of randomness are rarely separated: *which tasks* were sampled, *which run* of a stochastic agent was recorded, and *how* the agent and the task interact. Work on error bars for language-model evaluations (Miller, 2024; Bowyer et al., 2025), on variance in benchmarks (Madaan et al., 2024; Bouthillier et al., 2021) and on reliability metrics for agents (Yao et al., 2024; Kapoor et al., 2024) has established the principle. What is missing is an agent-specific toolkit that (i) treats the task as the sampling unit while exploiting repeated runs, (ii) decomposes variance across agents, tasks, their interaction and run noise, (iii) tells a practitioner how to split a budget between more tasks and more repeats, and (iv) is validated against ground truth.

Hypothesis. *Many differences between agents on published leaderboards are not statistically distinguishable, and metrics such as pass@k hide unreliability that pass^k reveals.* We treat this as a hypothesis to test, not an assumption, and report where the data support it and where they do not.

Contributions.

1. **AgentAudit**, a typed, tested Python library with a CLI and a browser playground that runs the same code (Pyodide). Statistics are implemented from scratch and cross-checked against SciPy and statsmodels (Section 3).

2. **AgentAudit-Bench**: six offline deterministic environments, 360 tasks with horizon and distractor knobs, verification by final state, seeded and logged tool failures, and record/replay caching (Section 4).

3. **An empirical study** with three legs: simulations with agents of known skill (coverage, error rates, power, allocation, sequential stopping, ranking calibration); a re-analysis of public SWE-bench Verified and tau-bench results; and an evaluation of 7 local open-weight models on 120 tasks with 10 runs each(Section 6).

4. **Practical rules** derived from the variance decomposition: a design effect $1 + (R-1)\rho$ for pooled intervals, a closed form for the cost-optimal number of runs per task, and a quantitative comparison of confidence sequences with fixed-sample tests and naive peeking.

Findings in brief. (1) Pooling runs is not merely loose: ten runs per task push the coverage of a nominal 95% pooled Wilson interval to 68.3%, while task-level bootstrap and hierarchical beta-binomial intervals stay within 92.5%–95.1%. (2) Pooled run-level tests have type-I error of 17.9% (nominal 5%) when agent-by-task interaction is large; task-level sign-flip tests stay in 2.2%–5.0%. (3) At a fixed budget of 1200 attempts, power falls from 68.7% with one run per task to 47.9% with ten. (4) On SWE-bench Verified, 28.3% of pairs are indistinguishable and no adjacent pair is separated. (5) A confidence sequence controls false positives where naive peeking does not (1.5% versus 36.5% under the null) but costs power against a fixed-sample test at the same budget. (6) On local models, run noise is 34% of variance, the best model's pass^{10} is 40.8% against a one-run score of 63.7%. 2 of the 7 models are near the floor (below 10%), where differences are trivially separable but uninformative about agent quality.

2 Related Work

Statistics for LLM evaluation. Miller (2024) gives standard errors, clustered errors, paired differences and power formulae for single-run question-answering evals; Bowyer et al. (2025) show that central-limit intervals fail for small benchmarks and recommend Bayesian or exact alternatives; Madaan et al. (2024) quantify seed variance for model training and evaluation; Biderman et al. (2024) document reproducibility pitfalls. Earlier work in NLP and RL made the same case for significance testing, power and robust aggregation (Dror et al., 2018; Card et al., 2020; Agarwal et al., 2021; Bouthillier et al., 2021). We build on this literature but target agents: several attempts per task, long-horizon outcomes verified by state, and the question of how to spend a budget.

Agent evaluation and reliability. Tau-bench introduced pass^k to measure consistency across trials (Yao et al., 2024); Kapoor et al. (2024) argued that agent benchmarks must account for cost and reproducibility; HAL standardised large-scale agent evaluation (Kapoor et al., 2026). Huang (2026) propose an estimand-aware pairwise procedure for agent leaderboards, and find unresolved close rank differences on SWE-bench and tau2-bench; Neuhof & Benjamini (2026) build rank intervals for MMLU from pairwise tests and analyse the sources of ranking uncertainty across subjects. Our contributions are complementary: a crossed variance decomposition over agents, tasks and runs, closed-form allocation of a budget between tasks and repeats, validation against synthetic ground truth, an offline benchmark with controlled stochasticity, and an all-pairs re-analysis with multiplicity control.

Anytime-valid inference. Confidence sequences (Howard et al., 2021) and betting-based constructions for bounded means (Waudby-Smith & Ramdas, 2024) permit continuous monitoring. Recent work applies them to LLM evaluation and online ranking (Hsu & Shekhar, 2026; Gu et al., 2026). We use the betting confidence sequence for paired per-task differences and measure, in simulation, what it costs and what it buys relative to fixed-sample tests and to naive peeking.

3 Preliminaries and Methodology

Setting. Agents $m = 1, \ldots, M$ are evaluated on tasks $t = 1, \ldots, T$ with R runs each, producing scores $Y_{mtr} \in [0, 1]$ (binary in our benchmark). We regard tasks as an i.i.d. sample from a task population and runs as conditionally independent given agent and task. All inference below is about that population; Section 8 discusses what this does and does not license.

3.1 pass@k and pass^k

With c successes in n attempts of one task, $\binom{c}{k}/\binom{n}{k}$ is an unbiased estimator of p^k (all k attempts succeed), and $1-\binom{n-c}{k}/\binom{n}{k}$ is unbiased for $1-(1-p)^k$ (Chen et al., 2021). We report the benchmark-level means of these per-task estimators; the plug-in $\hat{p}^{\,k}$ is biased upwards. pass^k is the reliability metric of tau-bench (Yao et al., 2024); it decreases in k whereas pass@k increases.

3.2 Intervals for a benchmark score

Let p_t be the success probability of task t, with mean p and variance τ^2 across tasks. The variance of the benchmark mean is

$$\mathrm{Var}(\bar{Y}) = \frac{\tau^2}{T} + \frac{p(1-p)-\tau^2}{TR}, \tag{1}$$

while a binomial interval on the pooled TR attempts assumes $p(1-p)/(TR)$. Their ratio is the design effect

$$\mathrm{DEFF} = 1+(R-1)\rho, \qquad \rho = \tau^2/\{p(1-p)\}, \tag{2}$$

so a nominal 95% pooled interval has coverage $2\Phi(1.96/\sqrt{\mathrm{DEFF}})-1$. We implement Wilson (Wilson, 1927), Clopper–Pearson (Clopper & Pearson, 1934) and Beta-Binomial credible intervals (valid for a single task or i.i.d. attempts), the *task bootstrap* (resample whole tasks; percentile and BCa (Efron, 1987; Efron & Tibshirani, 1993)), a *runs-only* bootstrap (ignores τ^2/T), a *two-stage* bootstrap (tasks, then runs; counts run noise twice), and an overdispersed beta-binomial profile-likelihood interval for the mean of $p_t \sim \mathrm{Beta}(\mu\kappa, (1-\mu)\kappa)$.

3.3 Variance decomposition

We fit the crossed random-effects model $Y_{mtr} = \mu + a_m + b_t + (ab)_{mt} + e_{mtr}$ with variances $\sigma^2_m, \sigma^2_t, \sigma^2_{mt}, \sigma^2_e$ by the ANOVA method of moments (expected mean squares in Appendix A), in the spirit of generalizability theory (Cronbach et al., 1972; Brennan, 2001). For binary scores this is a linear-probability decomposition: exact for variances of means, not latent-scale. Two derived quantities matter in practice: the variance of the difference between two agents,

$$\mathrm{Var}(\bar{Y}_a - \bar{Y}_b) = 2\Big(\frac{\sigma^2_{mt}}{T} + \frac{\sigma^2_e}{TR}\Big), \tag{3}$$

in which the task main effect cancels, and the generalizability coefficient $E\rho^2 = \sigma^2_m/(\sigma^2_m + \sigma^2_{mt}/T + \sigma^2_e/(TR))$, the share of observed between-agent variance that is true skill.

3.4 Paired comparison and multiplicity

For agents a, b let $d_t = \bar{Y}_{at} - \bar{Y}_{bt}$. The task is the unit of analysis. We provide a sign-flip permutation test (exact up to 14 non-zero differences), a cluster bootstrap for the mean difference, exact McNemar (McNemar, 1947) on per-task majority outcomes (the natural test when $R=1$), Cohen's d_z and the probability of superiority $P(d>0) + \frac{1}{2}P(d=0)$. Pairwise p-values are adjusted by Holm (Holm, 1979) or Benjamini–Hochberg (Benjamini & Hochberg, 1995).

3.5 Ranking uncertainty

We resample tasks (the same tasks for every agent), recompute scores and ranks (ties broken uniformly at random) and report rank distributions, $P(\text{top-}k)$, the matrix of $P(\bar{Y}_i > \bar{Y}_j)$, and the fraction of pairs that cannot be told apart. A split-half Kendall τ measures whether a fresh task sample would reproduce the ordering.

3.6 Power and the tasks-versus-repeats trade-off

Let $s_t = 2\sigma^2_{mt}$ and $s_e = 2\sigma^2_e$. The mean paired difference has $\mathrm{Var} = (s_t + s_e/R)/T$. At a fixed number of attempts $B = TR$ this equals $(Rs_t + s_e)/B$, *increasing* in R: if tasks are free, a fixed number of attempts is

best spent on more tasks. With cost c_{task} per new task and c_{run} per run, minimising the variance at budget $T(c_{\text{task}} + Rc_{\text{run}})$ gives

$$R^\star = \sqrt{\frac{s_e\, c_{\text{task}}}{s_t\, c_{\text{run}}}}. \tag{4}$$

Power uses the noncentral t distribution with $T-1$ degrees of freedom; the planner also reports the minimum detectable effect. Repeats remain essential for reliability metrics and for estimating σ_e^2 itself.

3.7 Anytime-valid sequential evaluation

For per-task quantities $X_t \in [0,1]$ (we use $(d_t+1)/2$) with mean m, the capital process $K_t(m) = \prod_{i\le t}(1 + \lambda_i(X_i - m))$ with predictable bets λ_i is a nonnegative martingale under the hypothesis that the mean equals m. By Ville's inequality, $\{m : K_t(m) < 1/\alpha\}$ is a confidence sequence valid at all times simultaneously (Howard et al., 2021; Waudby-Smith & Ramdas, 2024). We hedge both signs and use the predictable plug-in bet. A comparison stops when the sequence excludes a margin band or lies inside it.

4 AgentAudit-Bench

Environments. Six simulated worlds, all offline and deterministic: a virtual *file system*; an in-memory *SQLite* database; a REST-style *CRM* (customers and tickets); room *bookings*; *web forms* with validation and option codes; and a *tabular data* world with a restricted calculator. Each exposes 4–7 tools; tool errors are returned as observations, so an agent can recover from them.

Tasks. A procedural generator creates 360 tasks (60 per family) from a seed. Each family has five templates (for example, "move every `.txt` file into `/archive`", "upgrade every customer in a city"), and two knobs sweep a factorial grid: *horizon* (1–4 required operations or, for forms, 1–3 forms) and *distractors* (0–2: confusable files, decoy tables, same-surname customers, extra forms, extra rows). Free-text values are quoted in instructions to avoid punctuation ambiguity, a flaw we found when auditing early agent traces. A balanced one-third *core* subset (120 tasks, 20 per family) is used for the local-model evaluation.

Verification. Success is decided by the final world state or the submitted answer, never by the agent's text. Every task is audited automatically: a reference *oracle* must solve it, a do-nothing agent must not, and generation and resulting state must be deterministic in the seed.

Controlled stochasticity. A fault injector fails each tool call independently with probability q using a generator seeded by (task, run) and independent of the agent, so every model faces the same failures on the same attempt; a failed call leaves the world unchanged and returns a transient 503. Injections are logged. This turns reliability under tool failure into a controlled factor.

Agents and reproducibility. Agents implement a small interface. All local models use one *uniform text tool protocol*: tools are listed as JSON in the system prompt, a call is any JSON object with `name` and `arguments`, one call per turn is executed, and generation stops after it. Using one protocol removes chat-template differences from the comparison; it also avoids a defect we observed in the server-side parser of Ollama 0.40.8, which silently discarded well-formed calls from qwen2.5:3b when eight tools were declared (Appendix B). A record/replay cache keyed by agent configuration, message history, tools and seed makes every experiment replayable without the model. The runner executes attempts in run-major order, so any prefix of an interrupted evaluation is balanced, and resumes from checkpoints.

5 Experimental Protocol

Simulation studies. Synthetic agents follow a logit-normal item-response model, $p_{mt} = \sigma(\theta_m - b_t + g_{mt})$ with $b_t \sim N(0, \sigma_b^2)$ and agent-by-task interaction $g_{mt} \sim N(0, \sigma_g^2)$, and $Y_{mtr} \sim \text{Bernoulli}(p_{mt})$ independently across runs. Population means, pass^k and differences are computed by Gauss–Hermite quadrature, so coverage,

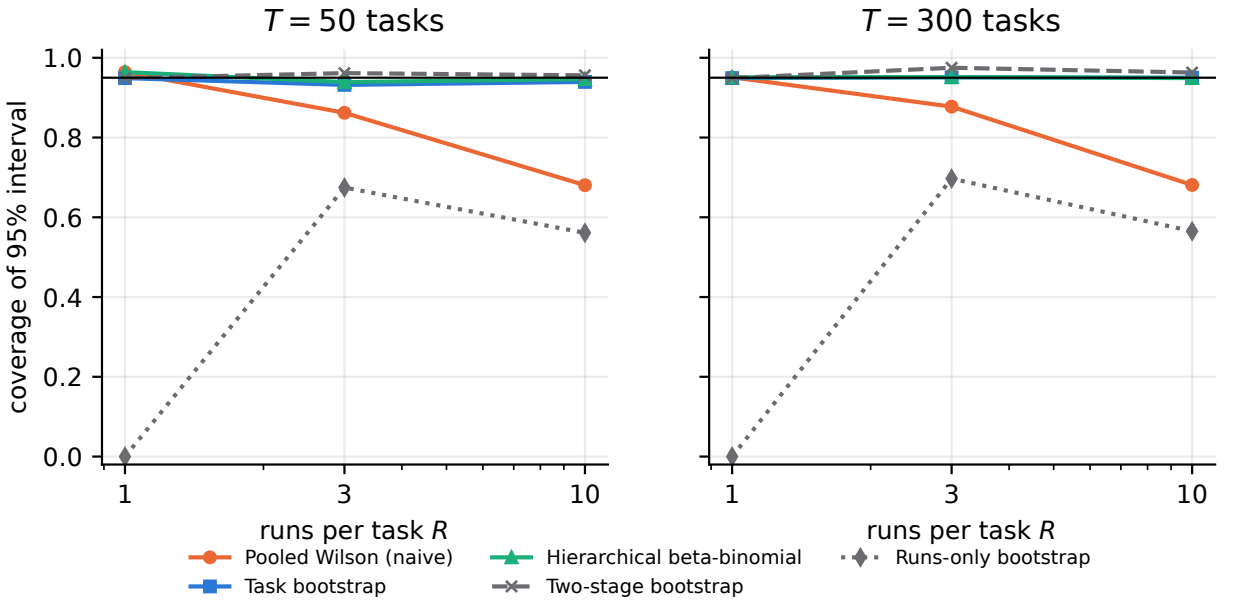


Figure 1: Coverage of nominal 95% intervals for the benchmark score as runs per task grow ($\sigma_b = 1.5$, interaction sd 0.5, true score ≈ 0.43). Pooling all attempts breaks down already at three runs.

error rates and power are measured against exact truth. Coverage cells use 2000 replications (Monte Carlo error at most about 1 point); all random streams are seeded and the scripts are in `scripts/sim_*.py`.

Public re-analysis. SWE-bench Verified: per-instance resolved lists of every submission in the public `swe-bench/experiments` repository that declares at least 50 resolved instances (one run per task, 500 instances), analysed with exact McNemar tests and Holm/Benjamini–Hochberg adjustment. The files do not state evaluation coverage; we assume each submission was evaluated on all 500 instances (Section 8). Tau-bench: the historical trajectories of gpt-4o and claude-3.5-sonnet (4 trials per task) released by the authors under the MIT license. Only derived aggregates are stored in this repository.

Local models. We evaluate 7 open-weight models served by Ollama on a single laptop (Apple M5, 16 GB) on the 120-task core subset of AgentAudit-Bench, 10 independent runs per task (8,400 attempts), temperature 0.7, with the text tool protocol of Appendix C so that every model is driven identically.

6 Results

6.1 Simulation: validating the statistics

Interval coverage. Figure 1 shows coverage of nominal 95% intervals for the benchmark score. With one run per task the pooled binomial intervals are fine (they are exact in that case). With three runs they cover 88.5% of the time and with ten runs only 68.3% (100 tasks), as predicted by the design effect. A runs-only bootstrap is worse (56.4%). The task bootstrap, its BCa variant and the hierarchical beta-binomial interval stay close to nominal (task bootstrap: 94.8% at ten runs; BCa 94.8%; beta-binomial 95.1%), and the two-stage bootstrap is slightly conservative (96.4%). Coverage of the task bootstrap for pass^k ranges between 90.8% and 94.8%, lower at 20 tasks, where the percentile bootstrap is known to be optimistic. Full numbers are in Appendix D.

Error rates of paired tests. With agent-by-task interaction sd = 1 and ten runs per task, pooled tests that treat each run as an independent pair reject a true null in 14.3% (two-proportion z-test) and 17.9%

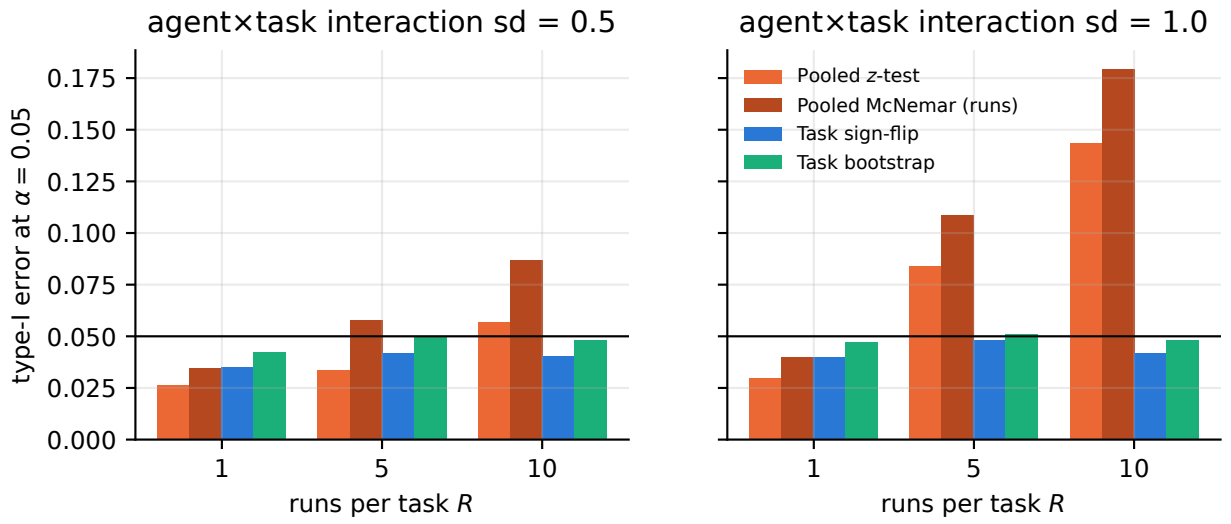


Figure 2: Type-I error at $\alpha = 0.05$ for $T = 100$ tasks and two agents of equal skill.

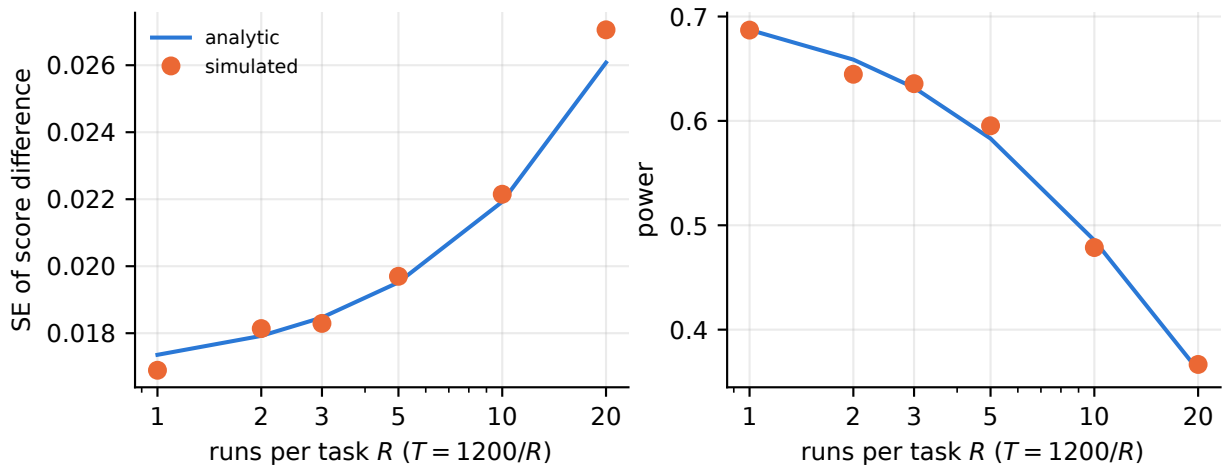


Figure 3: Fixed budget $B = 1200$ attempts, $T = B/R$ tasks. Left: standard error of the difference between two agents; right: power of the paired t-test.

(run-level McNemar) of simulations, whereas the task-level sign-flip and bootstrap tests reject in 4.2% and 4.8% (Figure 2). With smaller interaction (sd 0.5) the run-level McNemar test is still liberal (8.7%). The pooled z-test is not always anti-conservative (it ignores pairing and is conservative at small R), which is why the harm of pooling is not a simple inflation but a loss of validity.

Tasks or repeats? At a fixed budget of 1200 attempts the standard error of the score difference grows by a factor of 1.60 between one and twenty runs per task, and power drops from 68.7% to 47.9% (ten runs) and 36.7% (twenty runs) (Figure 3). The analytic variance model of Section 3.6 predicts the simulated standard errors to within 3.8% across all budgets. Repeats are not useless: they identify run noise and enable reliability metrics, but for *ranking* agents on a fixed attempt budget, tasks are worth more.

Sequential evaluation. Applying a fixed-sample test after every task ("peeking") and stopping at the first $p < 0.05$ gives 36.5% false positives under the null, against 1.5% for the confidence sequence and 4.5% for

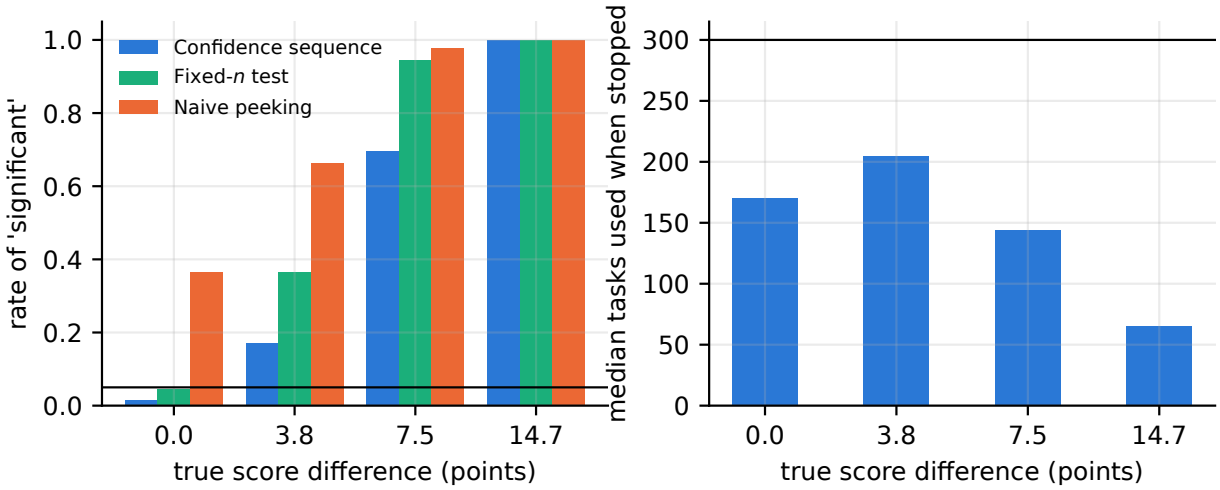


Figure 4: Sequential comparison, $R = 3$ runs per task. Left: fraction declared significant (the line marks 5%). Right: median number of tasks used when the sequence stops.

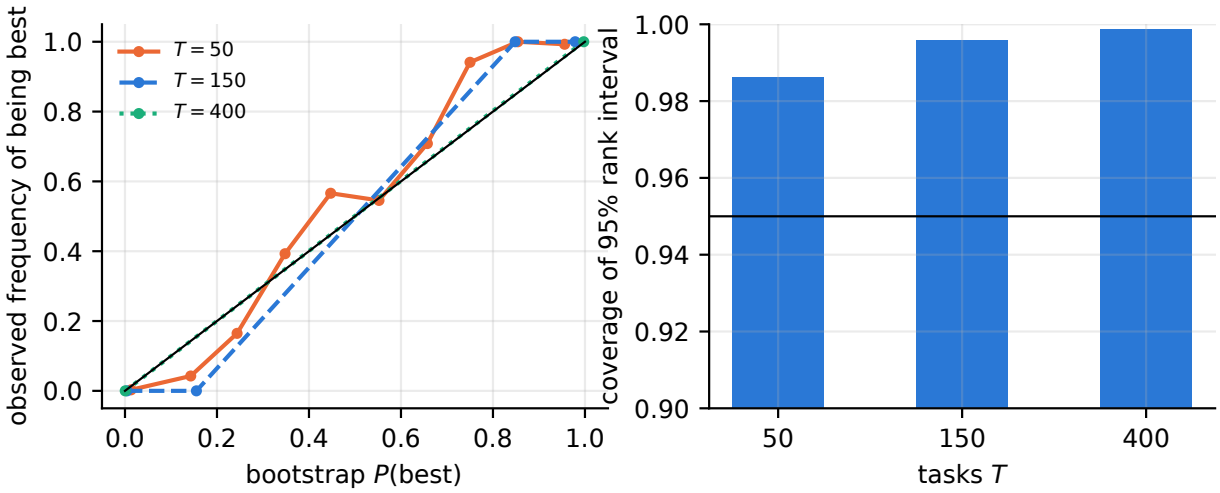


Figure 5: Six agents of known skill. Left: calibration of bootstrap P(best). Right: coverage of the 95% rank interval.

the fixed-n test (Figure 4). The price is power: with a true difference of 7.5 points the confidence sequence decides in 69.5% of runs against 94.5% for the fixed-n test at the full budget of 300 tasks. When the effect is large it stops early (median 65 tasks for the largest difference). Confidence sequences thus buy the right to stop at any time; they do not make evaluation cheaper at a given power.

Ranking uncertainty. Bootstrap rank intervals are conservative for the population ranking (coverage 98.6%–99.9% for nominal 95%), and P(best) is reasonably calibrated, noisily at $T = 50$ (Figure 5).

6.2 Re-analysis of public results

SWE-bench Verified. The 129 submissions yield 8 256 agent pairs. After Holm adjustment 71.7% are distinguishable at $\alpha = 0.05$ (86.4% with Benjamini–Hochberg; 86.7% without any correction), so 28.3% cannot be separated. The result is driven by the score gap (Figure 6): pairs less than five points apart are

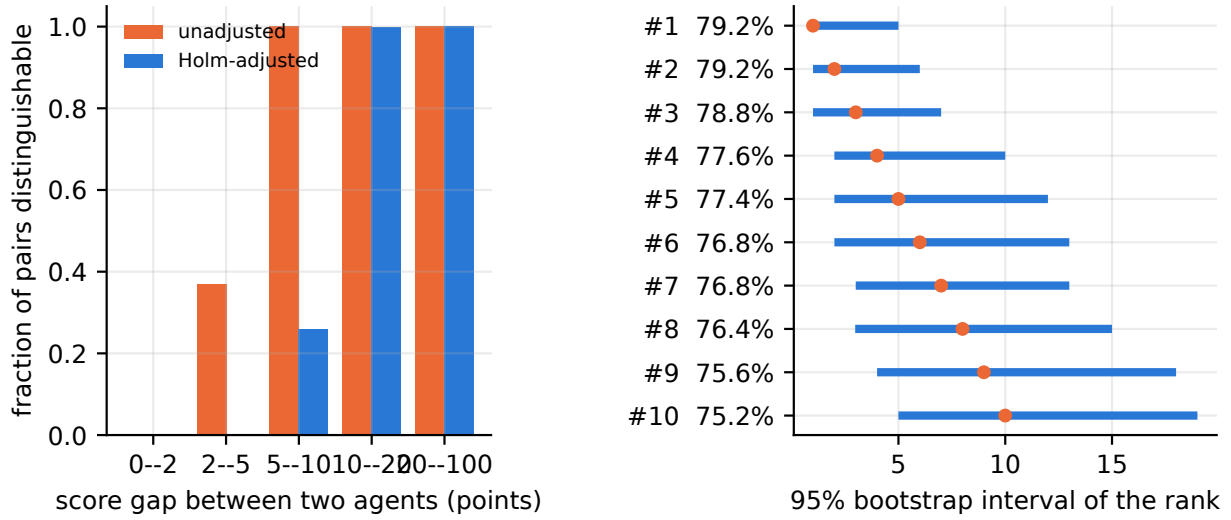


Figure 6: SWE-bench Verified, 129 public submissions. Left: fraction of pairs distinguishable by score gap. Right: top-10 scores with 95% bootstrap rank intervals.

essentially never separable after correction (0% for gaps of 2–5 points), pairs 5–10 points apart are separable in 26% of cases, and gaps above 10 points are separable almost always. The smallest distinguishable gap is 6.8 points and the median is 23.2. Among the 128 *adjacent* leaderboard positions, 0.0% are distinguishable after Holm over those comparisons (1.6% unadjusted). The top submission (79.2%) has a 95% rank interval of 1–5 and probability 34% of being best.

Tau-bench. For gpt-4o versus claude-3.5-sonnet with four trials per task, the retail gap is +10.2 points (95% interval [+3.9, +16.5]), task-level sign-flip $p = 0.002$; the pooled-runs z-test gives $p = 0.001$. In airline, the gap is +3.5 points [-6.0, +13.0] with $p = 0.545$ (pooled $p = 0.480$): not distinguishable with 50 tasks; at the observed effect about 787 tasks would be needed for 80% power (retail: 95). Variance is dominated by run noise (58% airline, 46% retail) and task difficulty (32%, 37%), with agent-by-task interaction of 10% and 15%. Figure 7 contrasts pass@k with pass^k: both models look strong when one success in k suffices and much weaker when all k must succeed.

6.3 Local open-weight models

Setup and headline numbers. 7 models, 120 tasks, 10 runs each (8 400 attempts). Success rates range from 0.0% (qwen2.5-3b-q2) to 63.7% (qwen2.5-7b); the full table with pass^k and per-family scores is in Appendix E. Figure 8 (left) shows 95% task-bootstrap intervals. The quantised variant `qwen2.5-3b-q2` never produced a parseable action (100% of its attempts end with no tool call), so its score of zero says nothing about the underlying agent; we keep it as a degenerate control rather than dropping it silently.

Reliability is much lower than the headline score. The best model's score of 63.7% falls to 52.9% for pass^3 and 40.8% for pass^{10} (middle panel): a model that solves a task on average does not solve it every time. Run noise accounts for 34% of the variance of single attempts, task difficulty for 14%, agent-by-task interaction for 26% and agent identity for 25%. Run noise is of the same order as the quantity of interest, so a one-run evaluation is a noisy measurement even here.

Ranking: the hypothesis is only partly borne out. Because the models differ a lot in capability, the ranking is well determined: 1 of 21 pairs are not separable after Holm correction (llama3.2-3b versus qwen2.5-3b, a gap of 1.7 points, adjusted $p = 0.67$), and split-half rank correlation is 0.94. This contrasts with the SWE-bench re-analysis, where scores are close; it shows that the statistical problems are most severe

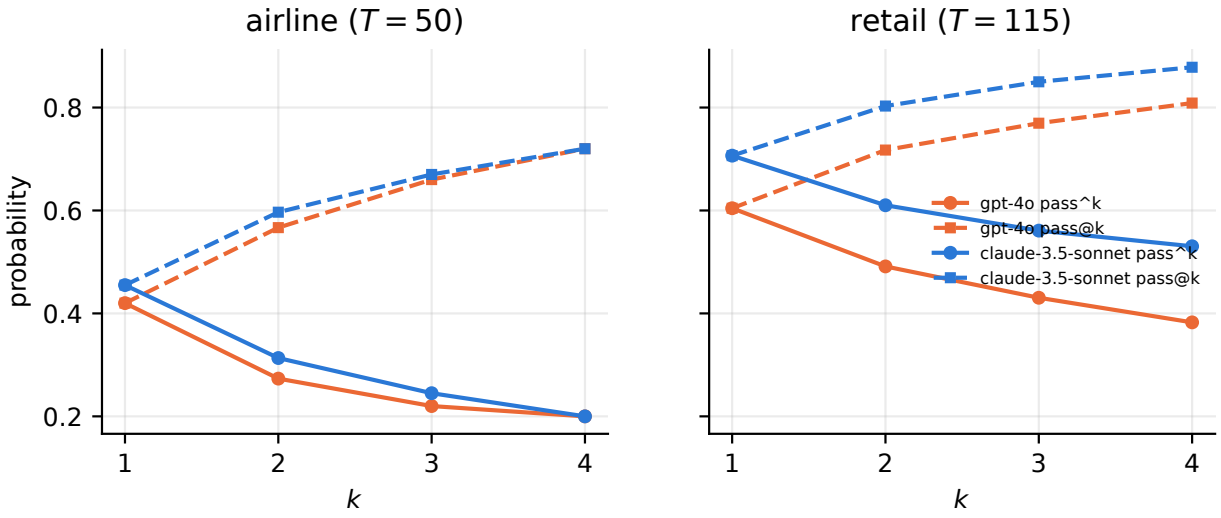


Figure 7: tau-bench, four trials per task: pass^k (solid) falls and pass@k (dashed) rises with k.

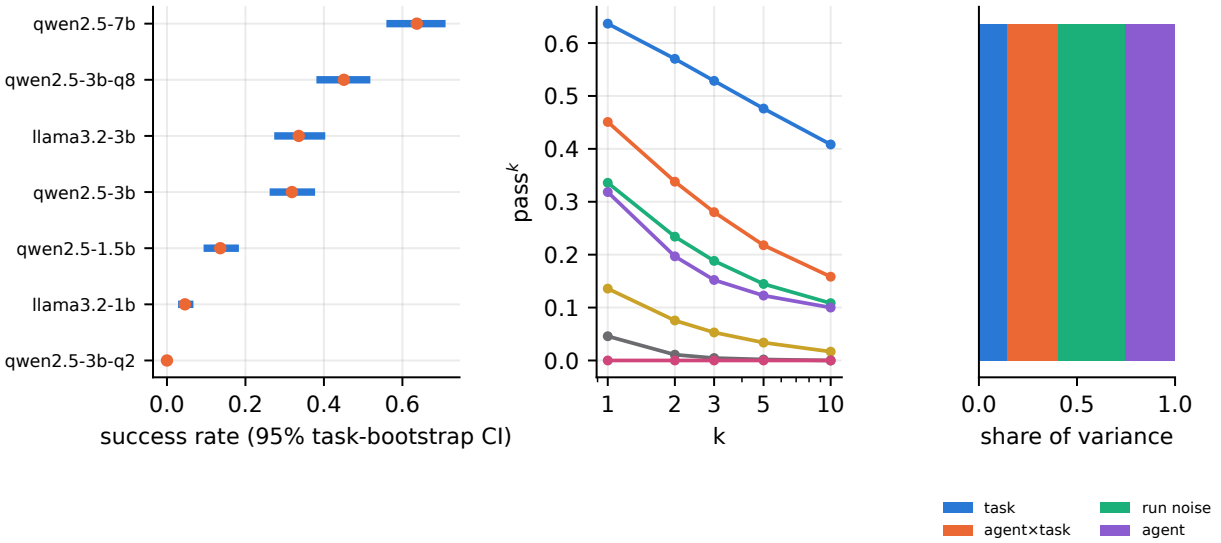


Figure 8: Local models on the 120-task core suite, 10 runs per task. Left: success rate with 95% task-bootstrap interval. Middle: pass^k. Right: variance decomposition of the full agents-by-tasks-by-runs array.

when systems are close, not that they are absent. The planner nevertheless says that resolving a 5-point difference between two of these agents at 80% power needs about 387 tasks at ten runs per task, more than three times the suite size; the minimum detectable difference of the current design is 9.0 points. With the cost model of Section 3.6 the optimal number of runs per task is only 4 when a task costs as much as ten runs.

6.4 Ablations on real data

Tasks versus runs. Sub-sampling the real data (Figure 9), a ranking from all 120 tasks with one run per task has Kendall $\tau = 0.98$ with the full-data ranking and separates 85% of the pairs, against 95% with ten runs. With only 12 tasks and one run the ranking correlation is $\tau = 0.86$ and only 12% of pairs are separated. Since the large gaps are easy, rank correlation saturates quickly; the number of *separated pairs* is

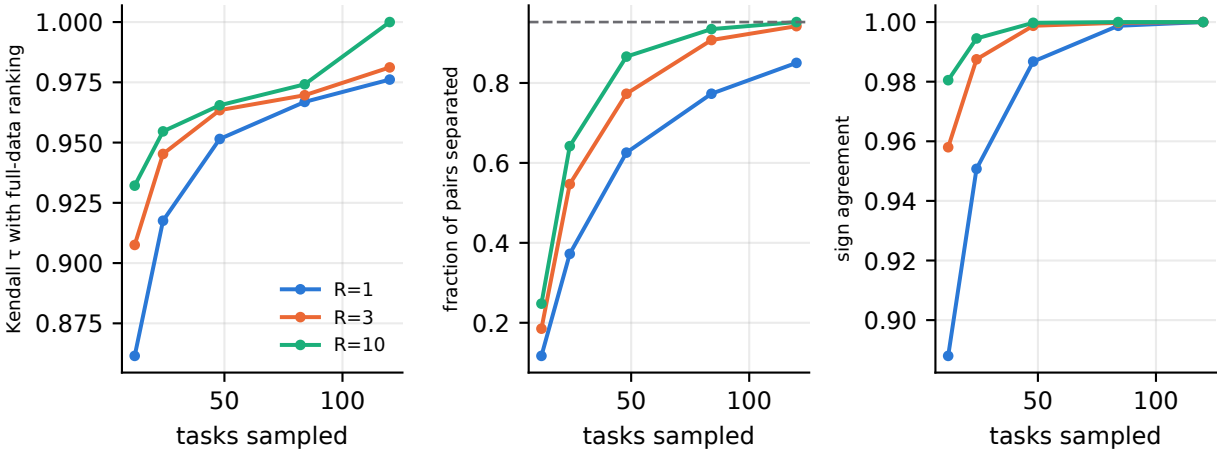


Figure 9: Sub-designs of the real data (random subsets of tasks and runs, 200 draws). Left: Kendall τ with the full-data ranking. Middle: fraction of the 21 pairs separated (paired t-test, Holm); the dashed line is the full data. Right: sign agreement for pairs that are significant in the full data.

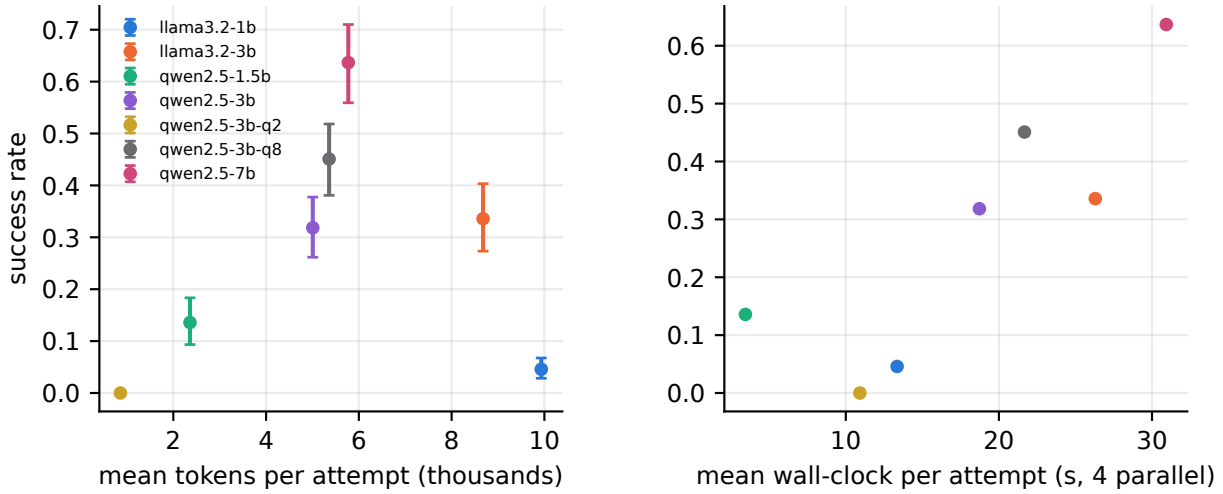


Figure 10: Success rate against mean tokens (left) and wall-clock (right) per attempt.

the informative measure, and it keeps growing with both tasks and runs. We did not repeat the fixed-budget comparison of Section 6.1 on real data, so the simulation result on tasks versus runs is not confirmed or refuted here.

Sequential stopping. Applying the confidence sequence to tasks in random order, 93% of the 20 pairs that are significant on the full data are decided before the tasks run out, with a median of 29 tasks. Large gaps are settled after a few tens of tasks; close pairs are not decided within the suite, as in the simulation. Here sequential stopping saves most of the compute, because most pairs differ a lot.

Cost profile. Figure 10 reports tokens and wall-clock per attempt. The weakest functioning model, llama3.2-1b, hits the step limit in 42% of attempts and consumes the most tokens: failure is not cheap, so cost per success varies more than cost per attempt.

7 Discussion

The simulation results are not surprising in isolation (clustered data need clustered inference), but their size is. Reporting a single pooled interval from several runs is common and, with ten runs per task, wrong by almost thirty coverage points. The public re-analysis shows that the consequence for leaderboards is concrete: with 500 instances and one run each, differences of up to about five points are routinely within noise, and the neighbours on the leaderboard are not separated. Neither finding says that leaderboards are useless: large gaps are reliably detected, and the top of the SWE-bench ranking is clearly above the bottom. They say that rank positions near each other should not be read as an ordering.

Recommendations. (i) Treat tasks as the sampling unit; report task-bootstrap or hierarchical intervals. (ii) Compare agents on the same tasks with paired task-level tests and correct for multiplicity. (iii) Plan with the variance decomposition: more tasks before more repeats, unless reliability (pass^k) is the target or tasks are expensive relative to runs. (iv) Report pass^k with pass@k. (v) If monitoring an evaluation as it runs, use a confidence sequence, not repeated fixed-sample tests. (vi) Publish per-task, per-run outcomes so that others can do this.

Refuted or weakened parts of the hypothesis. On local models the ranking is not fragile: with gaps of tens of points all but 1 of 21 pairs are separable and run noise does not reorder them, so the claim that leaderboard rankings are generally unreliable is supported only for close systems (SWE-bench) and not for widely spaced ones. We did not test the fixed-budget tasks-versus-runs result on real data, the real-data sequential analysis re-orders existing results rather than stopping a live run, and one model (q2) is a floor-effect control. Single hardware, single prompt protocol, small models: conclusions do not transfer to frontier agents.

8 Limitations, Ethics and Broader Impact

Limitations. The local-model evaluation uses small quantised models on one machine, so absolute scores and failure modes do not transfer to frontier agents; generality rests on the simulations and the public re-analysis. The environments are cleaner than real software tasks and checkers verify only the state the generator anticipates. Tasks are i.i.d. draws from the population defined by our generator, so intervals refer to that population, not to "ability". The variance decomposition is linear on the probability scale. The SWE-bench re-analysis assumes full coverage of the 500 instances by every submission and ignores differences in scaffolds, budgets and submission dates; the tau-bench data cover two models. Confidence sequences are conservative. Task difficulty in the benchmark is controlled by knobs we chose and is not calibrated to any human notion of difficulty.

Ethics and impact. No human subjects or personal data are involved; all environments are synthetic and all public data are used under their licenses (tau-bench: MIT; the SWE-bench experiments repository declares no license, so we store only derived aggregates and a download script). Better error bars can reduce overclaiming in agent evaluation, but statistical rigour can also be used to dismiss real improvements; a non-significant result is not evidence of equivalence, and we provide equivalence-margin stopping for that reason. All experiments used free, local, open-weight models and public data; no paid API was used.

9 Conclusion

Agent leaderboards carry less information than their precision suggests. A statistically principled treatment of tasks, runs and their interaction, tested against ground truth, shows where differences are real and tells practitioners how to spend an evaluation budget. We release the library, the benchmark, the playground and all scripts to regenerate every number in this paper.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2108.13264.

Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society Series B*, 57(1):289–300, 1995.

Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, et al. Lessons from the trenches on reproducible evaluation of language models. *arXiv preprint arXiv:2405.14782*, 2024.

Xavier Bouthillier et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 3, 2021. arXiv:2103.03098.

Sam Bowyer, Laurence Aitchison, and Desi R. Ivanova. Position: Don't use the CLT in LLM evals with fewer than a few hundred datapoints. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2503.01747.

Robert L. Brennan. *Generalizability Theory*. Springer, 2001.

Dallas Card, Peter Henderson, Urvashi Khandelwal, Robin Jia, Kyle Mahowald, and Dan Jurafsky. With little power comes great responsibility. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 9263–9274, 2020.

Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

Lee J. Cronbach, Goldine C. Gleser, Harinder Nanda, and Nageswari Rajaratnam. *The Dependability of Behavioral Measurements: Theory of Generalizability for Scores and Profiles*. Wiley, 1972.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1383–1392, 2018.

Bradley Efron. Better bootstrap confidence intervals. *Journal of the American Statistical Association*, 82 (397):171–185, 1987.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

Runzhe Gu, Wenguang Sun, Bowen Gang, and Xintao Xia. Anytime-valid inference for online ranking of large language models. In *Proceedings of the International Conference on Machine Learning (ICML), PMLR 306*, 2026.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2): 65–70, 1979.

Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, and Jasjeet Sekhon. Time-uniform, nonparametric, nonasymptotic confidence sequences. *The Annals of Statistics*, 49(2):1055–1080, 2021.

Chia-Yu Hsu and Shubhanshu Shekhar. Efficient sequential evaluation of large language models. *arXiv preprint arXiv:2607.17409*, 2026.

Wei-Jung Huang. What does an LLM-agent leaderboard rank actually compare? *arXiv preprint arXiv:2609.07785*, 2026.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.

Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, et al. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.11977.

Lovish Madaan, Aaditya K. Singh, Rylan Schaeffer, Andrew Poulton, Sanmi Koyejo, Pontus Stenetorp, Sharan Narang, and Dieuwke Hupkes. Quantifying variance in evaluation benchmarks. *arXiv preprint arXiv:2406.10229*, 2024.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. *arXiv preprint arXiv:2411.00640*, 2024.

Bitya Neuhof and Yuval Benjamini. Quantifying ranking uncertainty in LLM benchmarks. *arXiv preprint arXiv:2607.16259*, 2026.

Ian Waudby-Smith and Aaditya Ramdas. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B*, 86(1), 2024. doi: 10.1093/jrsssb/qkad009.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

A Mathematical details

For balanced data with M agents, T tasks and R runs, the expected mean squares of the crossed model are

$$E[MS_m] = \sigma_e^2 + R\sigma_{mt}^2 + TR\sigma_m^2, \qquad E[MS_t] = \sigma_e^2 + R\sigma_{mt}^2 + MR\sigma_t^2,$$
$$E[MS_{mt}] = \sigma_e^2 + R\sigma_{mt}^2, \qquad E[MS_e] = \sigma_e^2,$$

and the estimators solve these for the components, truncating negative values at zero (the raw values are retained). With $R = 1$ the interaction and error are confounded; the library flags this. For one agent, the one-way model gives $\hat{\sigma}_t^2 = (MS_b - MS_w)/R$ and $\hat{\sigma}_e^2 = MS_w$, which equals the REML estimate for balanced data (verified against statsmodels in the test suite).

Unbiasedness of pass^k. If $C \sim \mathrm{Bin}(n, p)$ then $E\binom{C}{k} = \binom{n}{k}p^k$, so $\binom{C}{k}/\binom{n}{k}$ is unbiased for p^k; similarly $E\binom{n-C}{k} = \binom{n}{k}(1-p)^k$.

Pooled design effect. Writing $\bar{Y} = \frac{1}{T}\sum_t \hat{p}_t$ with $\hat{p}_t \mid p_t \sim \mathrm{Bin}(R, p_t)/R$, the law of total variance gives $\mathrm{Var}(\bar{Y}) = \tau^2/T + E[p_t(1-p_t)]/(TR)$ and $E[p_t(1-p_t)] = p(1-p) - \tau^2$.

Optimal runs. Minimising $(s_t + s_e/R)(c_{\text{task}} + Rc_{\text{run}})$ over $R > 0$ gives $R^\star = \sqrt{s_e c_{\text{task}}/(s_t c_{\text{run}})}$; the planner evaluates the two neighbouring integers.

B Tool-protocol defect observed in Ollama

With Ollama 0.40.8 (macOS, Apple silicon), the native `/api/chat` tool-calling interface returned an empty message (no content, no tool calls) for `qwen2.5:3b` on a file-creation task when eight tools were declared, although the raw generation obtained from the `/api/generate` endpoint with the model's own template

contained a well-formed <tool_call> block. The failure was deterministic across seeds and temperatures; it disappeared with a single declared tool or without a system message. Because of this, all experiments use the uniform text protocol of Section 4. The observation is about one server version, not about the model.

C Prompts

System prompt.

```
You are an autonomous assistant that completes tasks by calling the provided tools. Work
step by step, use the tools to inspect the current state before changing it, and make
exactly the requested changes and no others. If a tool returns an error that looks
temporary (for example a 503), retry the call. When the task is done, call finish; if
the task asks a question, call submit_answer with the answer.
```

Tool section appended to the system prompt.

```
# Tools

You can call these tools (JSON signatures):
<tools>
{"name": "TOOL_NAME", "description": "DESCRIPTION", "parameters": {"type": "object",
    "properties": {"ARG": {"type": "string", "description": "DESCRIPTION"}}, "required":
    ["ARG"]}}
</tools>

To call a tool, reply with exactly one block of this form and nothing else:
<tool_call>
{"name": "<tool name>", "arguments": {<arguments as JSON>}}
</tool_call>
You will receive the result inside <tool_response></tool_response>. Call one tool at a
    time.
```

D Full coverage table

Table 1: Coverage (%) of nominal 95% intervals for the benchmark score; wilson, CP: pooled binomial intervals; the rest respect the task structure. T: tasks, R: runs per task.

T	R	Wilson	CP	Task boot.	BCa	Two-stage	Beta-bin.
20	1	94.9	94.9	91.2	94.0	91.2	94.9
20	3	87.9	89.6	93.3	94.7	96.5	95.0
20	10	68.2	70.2	92.5	94.2	95.2	93.8
50	1	96.4	96.4	94.9	96.2	94.9	96.4
50	3	86.2	87.9	93.2	93.6	96.2	93.9
50	10	68.0	69.2	94.0	94.5	95.6	94.7
100	1	94.4	96.0	94.3	94.8	94.3	94.4
100	3	88.5	88.5	94.9	95.1	96.8	95.0
100	10	68.3	68.3	94.8	94.8	96.4	95.1
300	1	95.1	95.1	94.9	95.2	94.9	95.1
300	3	87.8	88.9	95.1	95.2	97.5	95.2
300	10	68.1	68.8	95.0	95.0	96.3	94.9

Table 2: Local models, 120 tasks × 10 runs. Score, pass^3, pass^{10} in %; steps: mean agent turns; step-limit: share of attempts ending at the step budget; right columns: success rate (%) per task family.

Model	score	pass^3	pass^{10}	steps	step-limit	fs	sql	crm	booking	forms	tabular
qwen2.5-7b	63.7	52.9	40.8	6.0	2%	90	57	55	88	63	28
qwen2.5-3b-q8	45.1	28.0	15.8	4.7	1%	64	62	60	64	15	4
llama3.2-3b	33.6	18.8	10.8	8.5	22%	26	24	30	74	39	8
qwen2.5-3b	31.8	15.2	10.0	4.8	6%	56	25	36	52	12	10
qwen2.5-1.5b	13.6	5.3	1.7	2.9	0%	28	12	3	34	3	0
llama3.2-1b	4.6	0.5	0.0	8.5	42%	4	8	0	14	2	0
qwen2.5-3b-q2	0.0	0.0	0.0	1.0	0%	0	0	0	0	0	0

E Local-model results

F Reproducibility checklist

- Code, benchmark generator, configs and all scripts: `https://github.com/fragompul/agentaudit`; `make reproduce` regenerates simulation results, re-analyses and figures; every number in this paper is produced by `scripts/make_paper_assets.py` from `results/`.
- Seeds are fixed in every script; model tags and quantisations are in `configs/models.json`; the test suite runs without network access (coverage above 90%, `mypy -strict`).
- Raw trajectories and the record/replay caches are released as release assets (they are not stored in git).
- Compute: a single Apple M5 laptop with 16 GB unified memory; no cloud, no paid API.